

TPC-H ベンチマークレポート

Gen1 vs Gen2 XSmall ウェアハウス
データボリューム別性能・コスト比較

2026年6月26日

Snowflake アカウント: SFSEAPAC-K_FUKAMORI

1. 実験概要と条件

1.1 目的

Snowflake Gen1 (第1世代) と Gen2 (第2世代) のXSmallウェアハウスにおいて、データボリュームの違いがクエリ実行時間およびクレジット消費にどの程度影響するかを定量的に測定する。

1.2 実験条件

項目	設定値
ウェアハウス (Gen1)	BENCH_GEN1_XS (X-Small, STANDARD_GEN_1, 1.0 credit/h)
ウェアハウス (Gen2)	BENCH_GEN2_XS (X-Small, STANDARD_GEN_2, 1.35 credit/h)
データソース	SNOWFLAKE_SAMPLE_DATA (TPC-H)
スケールファクター	SF1, SF10, SF100, SF1000
ベンチマーククエリ	TPC-H Q1, Q3, Q5, Q6, Q9, Q12, Q18, Q21 (8種)
各クエリ実行回数	5回 (平均値を採用)
リザルトキャッシュ	OFF (USE_CACHED_RESULT = FALSE)
Query Acceleration	OFF (ENABLE_QUERY_ACCELERATION = FALSE)

1.3 クレジット単価

Gen1 XSmall: 1.0 credit/時間

Gen2 XSmall: 1.35 credit/時間

Gen2はGen1と比較してクレジット単価が35%高いが、より高速なハードウェアとソフトウェア最適化により実行時間が短縮されるため、実質コストの比較が重要となる。

2. データセット情報

TPC-Hベンチマークの4つのスケールファクターを使用。スケールファクターが10倍になるごとにデータ量もほぼ10倍に増加する。
注: 以下のサイズはSnowflake内部の圧縮後ストレージサイズ (INFORMATION_SCHEMA.TABLES.BYTES) 。

2.1 SF1 (合計: 0.243 GB)

テーブル	行数	サイズ (GB)
LINEITEM	6,001,215	0.154
ORDERS	1,500,000	0.039
PARTSUPP	800,000	0.034
CUSTOMER	150,000	0.010
PART	200,000	0.005
SUPPLIER	10,000	0.001
NATION	25	0.000
REGION	5	0.000

2.2 SF10 (合計: 2.519 GB)

テーブル	行数	サイズ (GB)
LINEITEM	59,986,052	1.599
ORDERS	15,000,000	0.416
PARTSUPP	8,000,000	0.348
CUSTOMER	1,500,000	0.101
PART	2,000,000	0.049
SUPPLIER	100,000	0.006
NATION	25	0.000
REGION	5	0.000

2.3 SF100 (合計: 24.875 GB)

テーブル	行数	サイズ (GB)
LINEITEM	600,037,902	15.465
ORDERS	150,000,000	4.326
PARTSUPP	80,000,000	3.508
CUSTOMER	15,000,000	1.011
PART	20,000,000	0.499
SUPPLIER	1,000,000	0.064
NATION	25	0.000
REGION	5	0.000

2.4 SF1000 (合計: 258.791 GB)

テーブル	行数	サイズ (GB)
LINEITEM	5,999,989,709	159.276
ORDERS	1,500,000,000	48.614
PARTSUPP	800,000,000	35.071
CUSTOMER	150,000,000	10.149
PART	200,000,000	5.037
SUPPLIER	10,000,000	0.644
NATION	25	0.000
REGION	5	0.000

3. 個別クエリ実行結果

以下、8つのTPC-Hクエリそれぞれについて、クエリ内容・処理特徴・実行結果を示す。

3.1 Q1 - Pricing Summary Report

クエリ内容

```
SELECT l_returnflag, l_linestatus,
       SUM(l_quantity) AS sum_qty,
       SUM(l_extendedprice) AS sum_base_price,
       SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
       SUM(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
       AVG(l_quantity) AS avg_qty, AVG(l_extendedprice) AS avg_price,
       AVG(l_discount) AS avg_disc, COUNT(*) AS count_order
FROM LINEITEM
WHERE l_shipdate <= DATEADD(day, -90, '1998-12-01')
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;
```

処理特徴

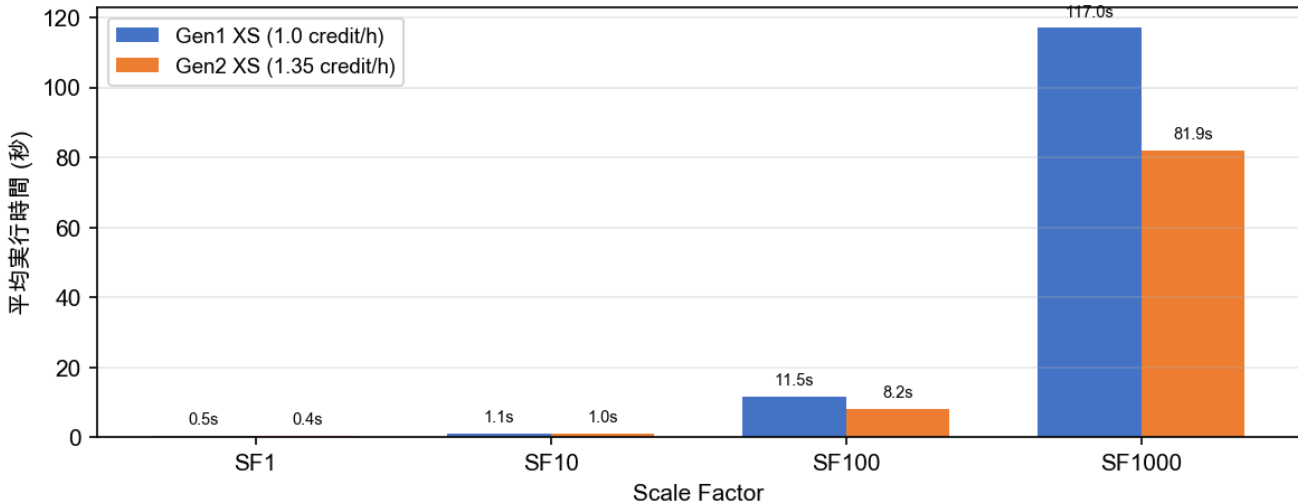
LINEITEM単体に対する大規模集計クエリ。出荷日によるフィルタ後、返品フラグとステータスでGROUP BYし、数量・金額・割引等の集計値を算出する。
処理特徴: フルテーブルスキャン + 集計演算が主体。
JOINなし。データ量に対してほぼニアにスケールする。

参照テーブル: LINEITEM

実行結果 (平均実行時間・秒, 5回平均)

Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.15	0.48	0.42	+12.5%	0.00013	0.00016
SF10	1.60	1.08	1.00	+7.4%	0.00030	0.00038
SF100	15.46	11.50	8.17	+29.0%	0.00319	0.00306
SF1000	159.28	116.98	81.87	+30.0%	0.03249	0.03070

Q1: スケール別実行時間比較



3.2 Q3 - Shipping Priority

クエリ内容

```
SELECT l_orderkey,
       SUM(l_extendedprice * (1 - l_discount)) AS revenue,
       o_orderdate, o_shippriority
FROM CUSTOMER, ORDERS, LINEITEM
WHERE c_mktsegment = 'BUILDING'
      AND c_custkey = o_custkey AND l_orderkey = o_orderkey
      AND o_orderdate < '1995-03-15' AND l_shipdate > '1995-03-15'
GROUP BY l_orderkey, o_orderdate, o_shippriority
ORDER BY revenue DESC, o_orderdate LIMIT 10;
```

処理特徴

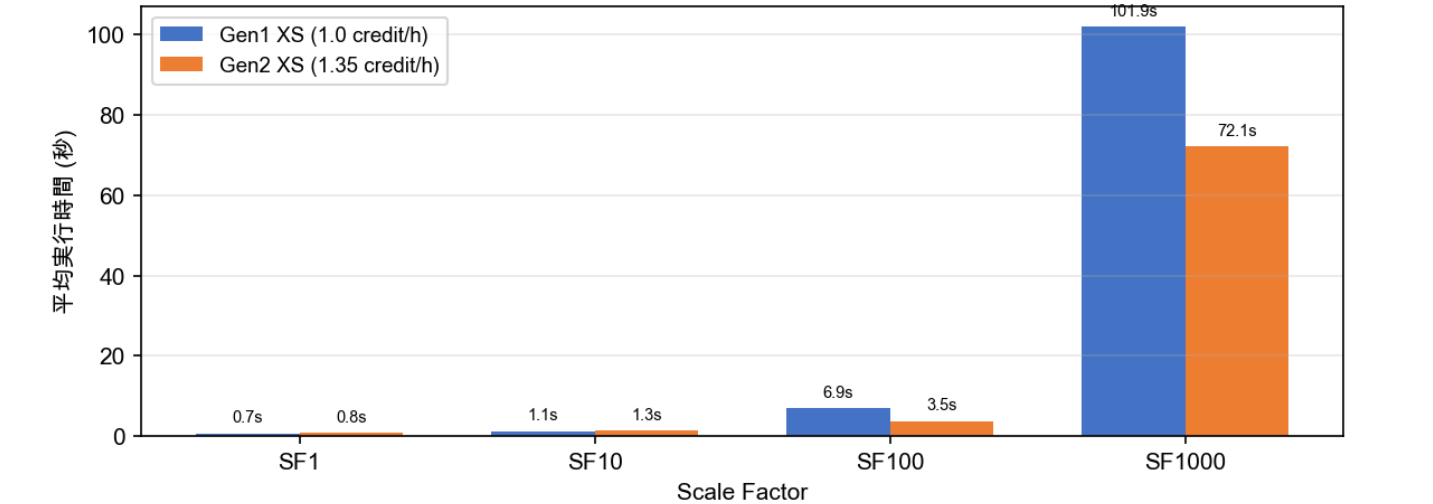
3テーブル(CUSTOMER, ORDERS, LINEITEM)をJOINし、特定市場セグメントの未出荷注文の収益をTOP10で取得。
処理特徴: 複数テーブルJOIN + フィルタ + 集計 + ソート。
JOIN順序の最適化がパフォーマンスに影響する典型例。

参照テーブル: CUSTOMER, ORDERS, LINEITEM

実行結果 (平均実行時間・秒, 5回平均)

Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.20	0.71	0.75	-5.6%	0.00020	0.00028
SF10	2.12	1.07	1.31	-22.4%	0.00030	0.00049
SF100	20.80	6.89	3.55	+48.5%	0.00191	0.00133
SF1000	218.04	101.89	72.07	+29.3%	0.02830	0.02703

Q3: スケール別実行時間比較



3.3 Q5 - Local Supplier Volume

クエリ内容

```
SELECT n_name,
       SUM(l_extendedprice * (1 - l_discount)) AS revenue
FROM CUSTOMER, ORDERS, LINEITEM, SUPPLIER, NATION, REGION
WHERE c_custkey = o_custkey AND l_orderkey = o_orderkey
      AND l_suppkey = s_suppkey AND c_nationkey = s_nationkey
      AND s_nationkey = n_nationkey AND n_regionkey = r_regionkey
      AND r_name = 'ASIA'
      AND o_orderdate >= '1994-01-01'
      AND o_orderdate < DATEADD(year, 1, '1994-01-01')
GROUP BY n_name ORDER BY revenue DESC;
```

処理特徴

6テーブル(CUSTOMER~REGION)をJOINし、アジア地域の
国別売上収益を算出。

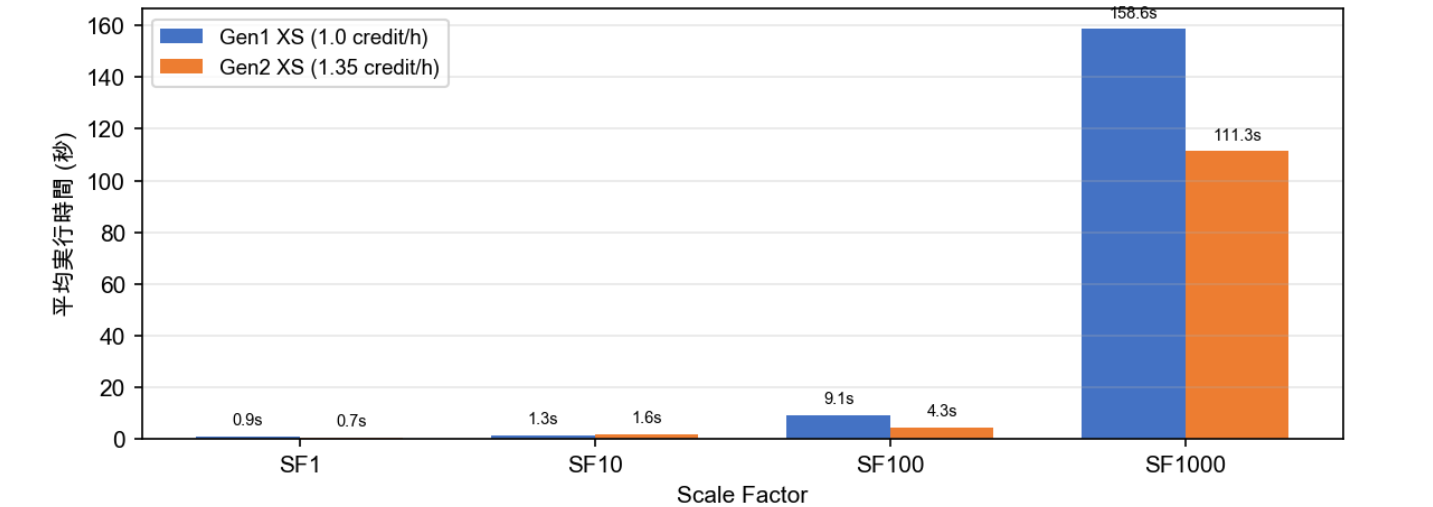
処理特徴: 多テーブルJOIN(6テーブル) + リージョン
フィルタ。JOIN数が多いため実行計画の選択が重要。

参照テーブル: CUSTOMER, ORDERS, LINEITEM, SUPPLIER, NATION, REGION

実行結果 (平均実行時間・秒, 5回平均)

Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.20	0.86	0.67	+22.1%	0.00024	0.00025
SF10	2.12	1.29	1.62	-25.6%	0.00036	0.00061
SF100	20.87	9.12	4.35	+52.3%	0.00253	0.00163
SF1000	218.68	158.60	111.31	+29.8%	0.04406	0.04174

Q5: スケール別実行時間比較



3.4 Q6 - Forecasting Revenue Change

クエリ内容

```
SELECT SUM(l_extendedprice * l_discount) AS revenue
FROM LINEITEM
WHERE l_shipdate >= '1994-01-01'
      AND l_shipdate < DATEADD(year, 1, '1994-01-01')
      AND l_discount BETWEEN 0.05 AND 0.07
      AND l_quantity < 24;
```

処理特徴

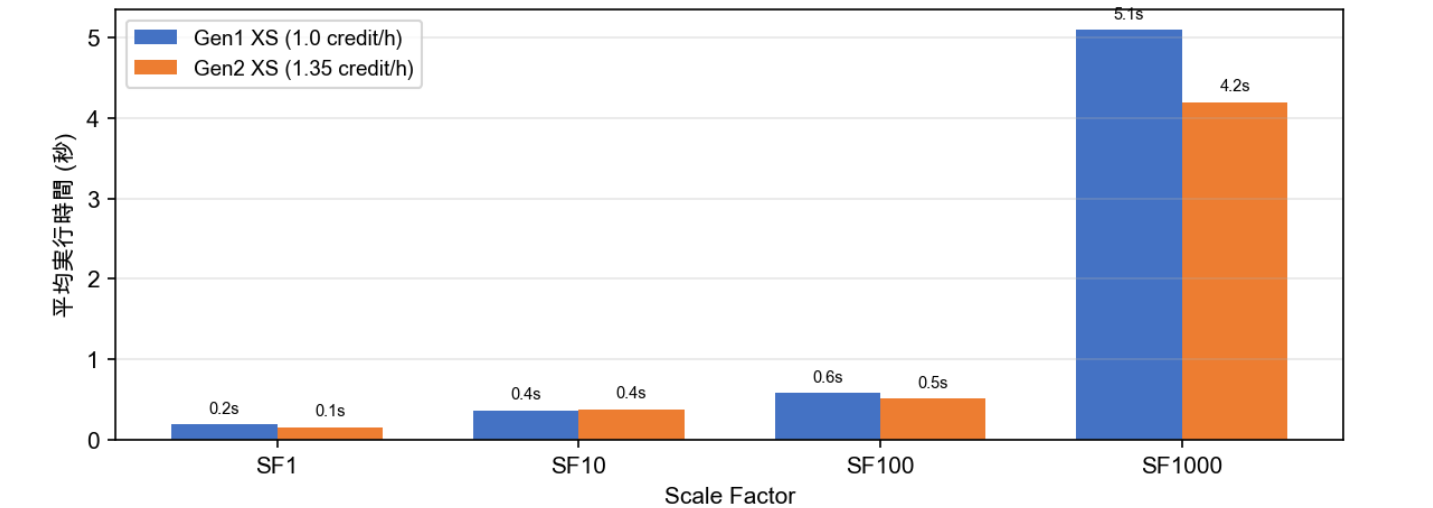
LINEITEM単体に対する単純なフィルタ+SUM集計。
出荷日・割引・数量の3条件でフィルタし売上予測を計算。
処理特徴: 最も単純なクエリ。マイクロパーティション
プルーニングが非常に効果的に働く。

参照テーブル: LINEITEM

実行結果 (平均実行時間・秒, 5回平均)

Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.15	0.19	0.15	+21.1%	0.00005	0.00006
SF10	1.60	0.37	0.38	-2.7%	0.00010	0.00014
SF100	15.46	0.58	0.51	+12.1%	0.00016	0.00019
SF1000	159.28	5.09	4.19	+17.7%	0.00141	0.00157

Q6: スケール別実行時間比較



3.5 Q9 - Product Type Profit Measure

クエリ内容

```
SELECT nation, o_year, SUM(amount) AS sum_profit
FROM (
  SELECT n_name AS nation, YEAR(o_orderdate) AS o_year,
    l_extendedprice*(1-l_discount) - ps_supplycost*l_quantity AS amount
  FROM PART, SUPPLIER, LINEITEM, PARTSUPP, ORDERS, NATION
  WHERE s_suppkey=l_suppkey AND ps_suppkey=l_suppkey
    AND ps_partkey=l_partkey AND p_partkey=l_partkey
    AND o_orderkey=l_orderkey AND s_nationkey=n_nationkey
    AND p_name LIKE '%green%'
) GROUP BY nation, o_year
ORDER BY nation, o_year DESC;
```

処理特徴

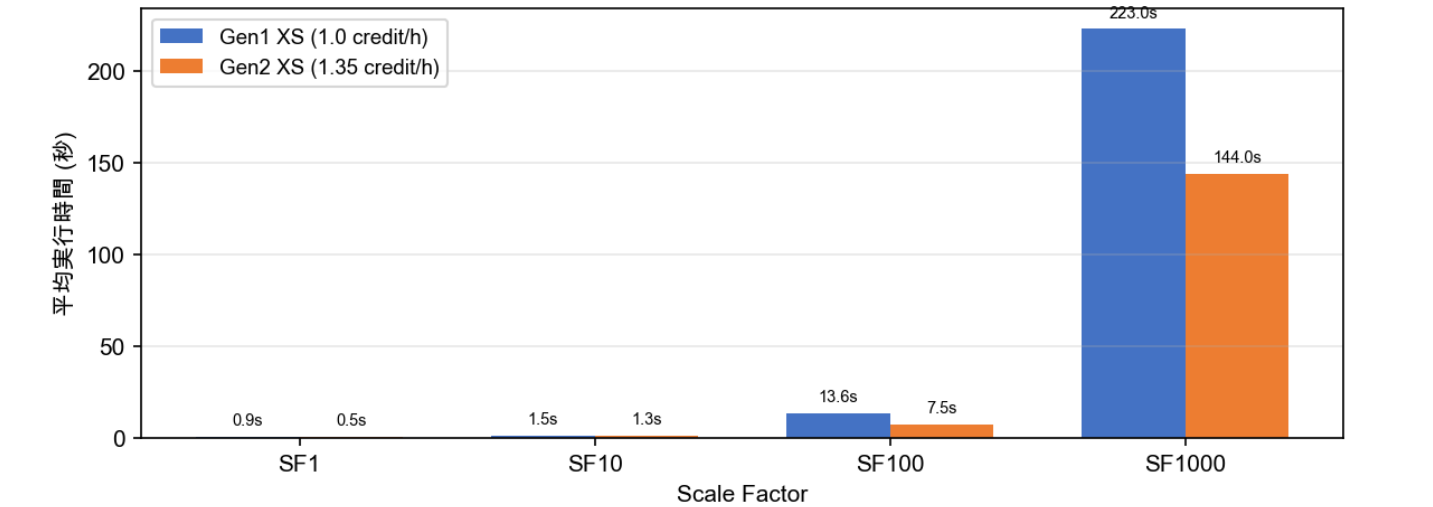
6テーブルをJOINし、製品名にLIKE条件を適用して
国別・年別の利益を集計する。
処理特徴: 多テーブルJOIN + LIKE述語(パターン検索) +
派生テーブル。LIKE検索とJOINの組み合わせが高コスト。

参照テーブル: PART, SUPPLIER, LINEITEM, PARTSUPP, ORDERS, NATION

実行結果 (平均実行時間・秒, 5回平均)

Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.23	0.89	0.48	+46.1%	0.00025	0.00018
SF10	2.42	1.48	1.29	+12.8%	0.00041	0.00048
SF100	23.86	13.61	7.51	+44.8%	0.00378	0.00282
SF1000	248.64	223.04	143.99	+35.4%	0.06196	0.05400

Q9: スケール別実行時間比較



3.6 Q12 - Shipping Modes and Order Priority

クエリ内容

```
SELECT l_shipmode,
  SUM(CASE WHEN o_orderpriority IN ('1-URGENT','2-HIGH')
    THEN 1 ELSE 0 END) AS high_line_count,
  SUM(CASE WHEN o_orderpriority NOT IN ('1-URGENT','2-HIGH')
    THEN 1 ELSE 0 END) AS low_line_count
FROM ORDERS, LINEITEM
WHERE o_orderkey = l_orderkey
  AND l_shipmode IN ('MAIL', 'SHIP')
  AND l_commitdate < l_receiptdate
  AND l_shipdate < l_commitdate
  AND l_receiptdate >= '1994-01-01'
  AND l_receiptdate < DATEADD(year, 1, '1994-01-01')
GROUP BY l_shipmode ORDER BY l_shipmode;
```

処理特徴

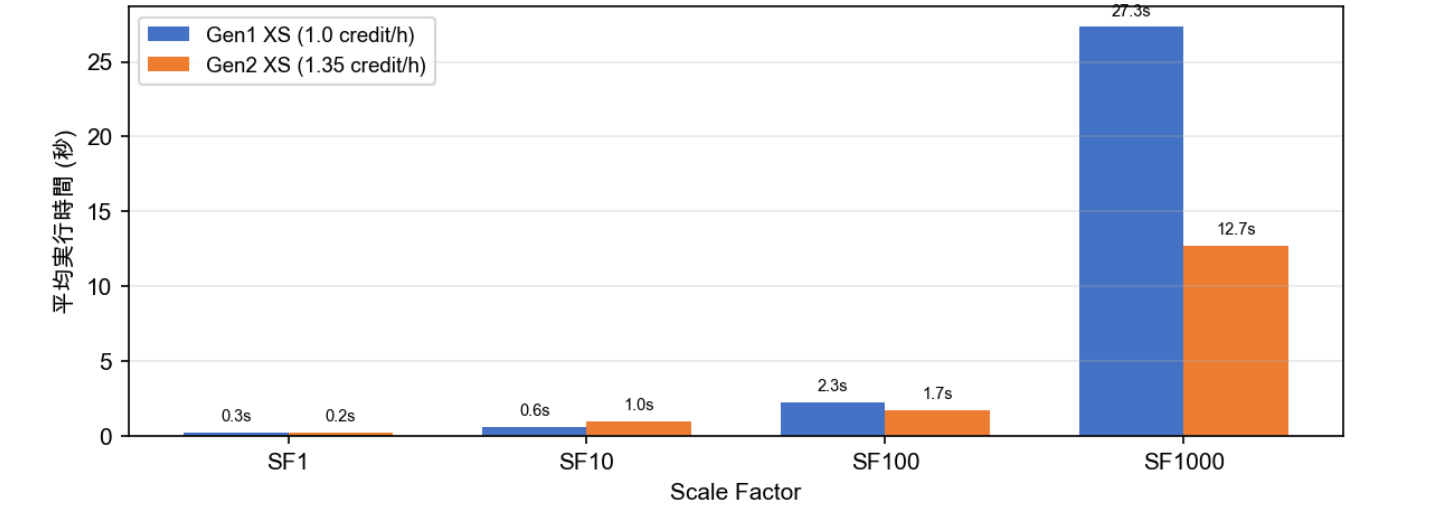
2テーブル(ORDERS, LINEITEM)をJOINし、配送モード別に注文優先度の高低を集計。
処理特徴: 2テーブルJOIN + 複数日付条件フィルタ + CASE式による条件集計。比較的軽量のJOINクエリ。

参照テーブル: ORDERS, LINEITEM

実行結果 (平均実行時間・秒, 5回平均)

Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.19	0.26	0.24	+7.7%	0.00007	0.00009
SF10	2.02	0.63	0.97	-54.0%	0.00017	0.00036
SF100	19.79	2.26	1.74	+23.0%	0.00063	0.00065
SF1000	207.89	27.32	12.71	+53.5%	0.00759	0.00477

Q12: スケール別実行時間比較



3.7 Q18 - Large Volume Customer

クエリ内容

```
SELECT c_name, c_custkey, o_orderkey,
       o_orderdate, o_totalprice, SUM(l_quantity)
FROM CUSTOMER, ORDERS, LINEITEM
WHERE o_orderkey IN (
  SELECT l_orderkey FROM LINEITEM
  GROUP BY l_orderkey HAVING SUM(l_quantity) > 300)
  AND c_custkey = o_custkey AND o_orderkey = l_orderkey
GROUP BY c_name, c_custkey, o_orderkey, o_orderdate, o_totalprice
ORDER BY o_totalprice DESC, o_orderdate LIMIT 100;
```

処理特徴

サブクエリで大量注文(数量>300)を特定し、該当する顧客・注文情報をTOP100で取得。

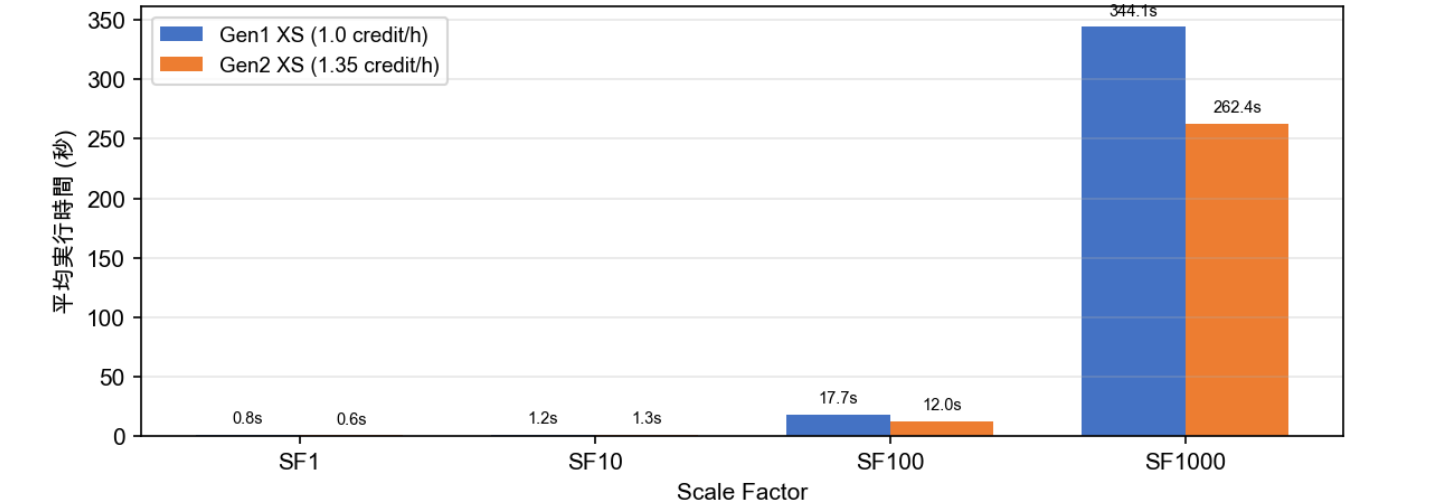
処理特徴: 相関サブクエリ + GROUP BY HAVING + ソート。サブクエリのコストが大規模データで顕著。

参照テーブル: CUSTOMER, ORDERS, LINEITEM

実行結果 (平均実行時間・秒, 5回平均)

Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.20	0.84	0.57	+32.1%	0.00023	0.00021
SF10	2.12	1.19	1.34	-12.6%	0.00033	0.00050
SF100	20.80	17.70	12.02	+32.1%	0.00492	0.00451
SF1000	218.04	344.08	262.45	+23.7%	0.09558	0.09842

Q18: スケール別実行時間比較



3.8 Q21 - Suppliers Who Kept Orders Waiting

クエリ内容

```
SELECT s_name, COUNT(*) AS numwait
FROM SUPPLIER, LINEITEM l1, ORDERS, NATION
WHERE s_suppkey = l1.l_suppkey
  AND o_orderkey = l1.l_orderkey AND o_orderstatus = 'F'
  AND l1.l_receiptdate > l1.l_commitdate
  AND EXISTS (SELECT * FROM LINEITEM l2
    WHERE l2.l_orderkey = l1.l_orderkey
    AND l2.l_suppkey <> l1.l_suppkey)
  AND NOT EXISTS (SELECT * FROM LINEITEM l3
    WHERE l3.l_orderkey = l1.l_orderkey
    AND l3.l_suppkey <> l1.l_suppkey
    AND l3.l_receiptdate > l3.l_commitdate)
  AND s_nationkey = n_nationkey AND n_name = 'SAUDI ARABIA'
GROUP BY s_name ORDER BY numwait DESC, s_name LIMIT 100;
```

処理特徴

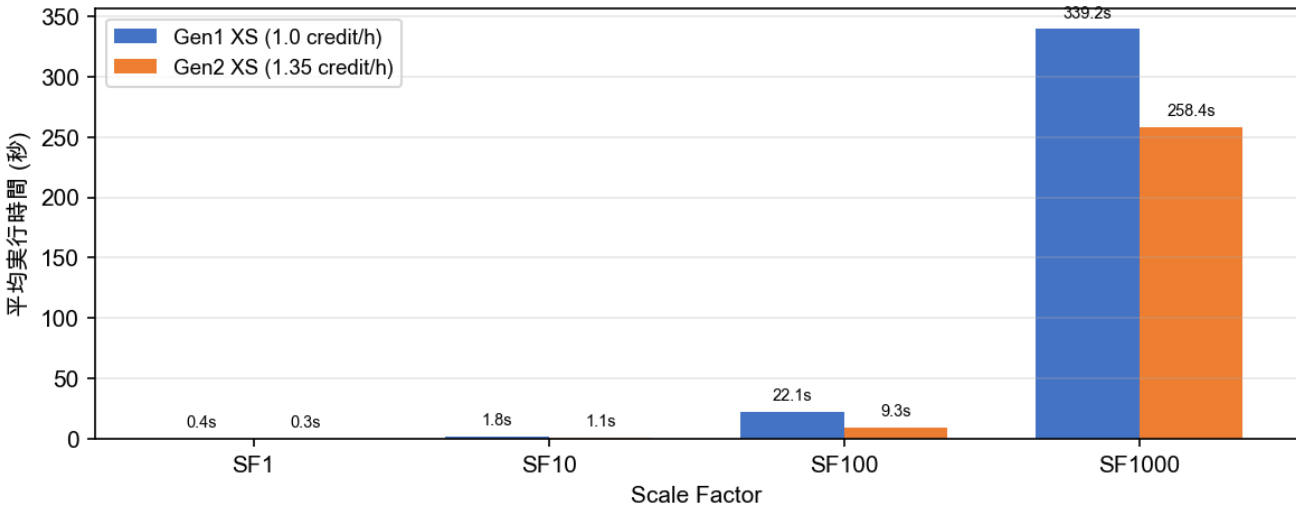
納品遅延のあるサプライヤを特定。EXISTS/NOT EXISTS
で同一注文内の他サプライヤの状況を確認する複雑クエリ。
処理特徴: 4テーブルJOIN + EXISTS + NOT EXISTS。
最も複雑なクエリで大規模データでの実行時間が最長。

参照テーブル: SUPPLIER, LINEITEM, ORDERS, NATION

実行結果 (平均実行時間・秒, 5回平均)

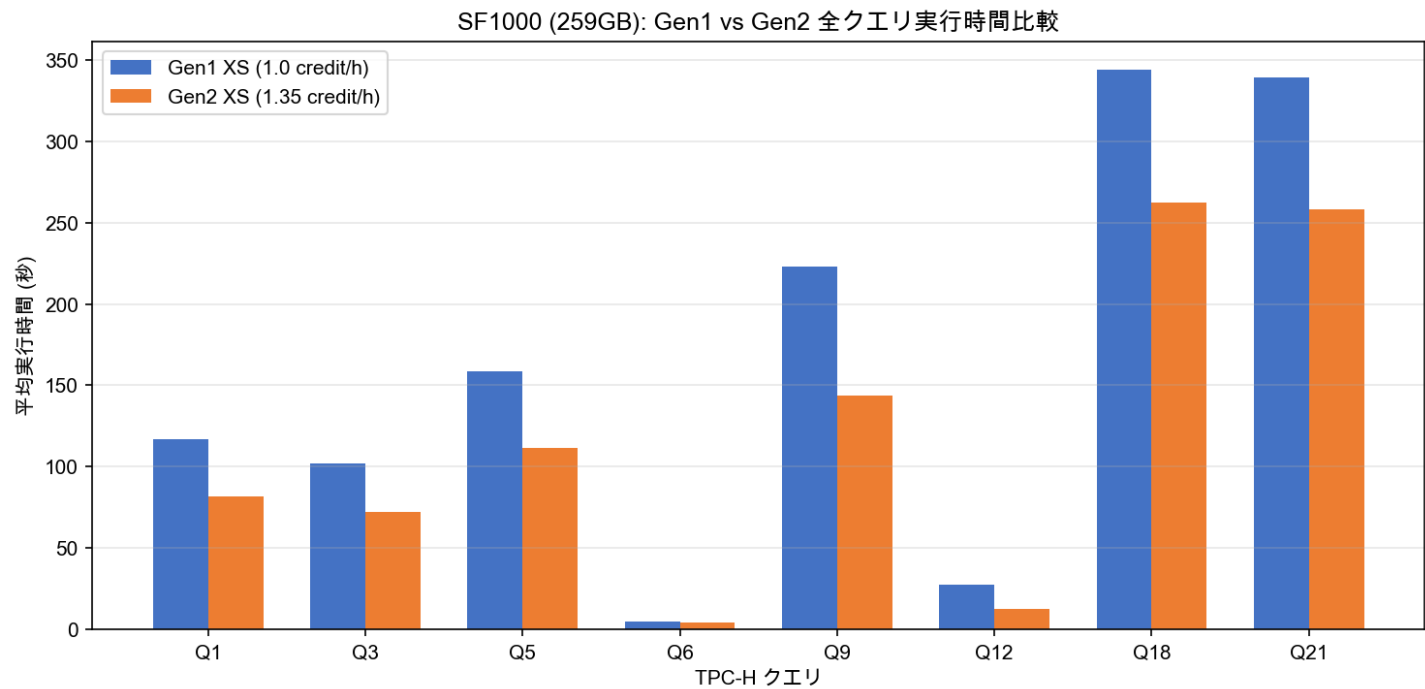
Scale Factor	参照データ量(GB)	Gen1 (秒)	Gen2 (秒)	改善率	Gen1コスト(credit)	Gen2コスト(credit)
SF1	0.19	0.42	0.35	+16.7%	0.00012	0.00013
SF10	2.02	1.77	1.14	+35.6%	0.00049	0.00043
SF100	19.86	22.06	9.31	+57.8%	0.00613	0.00349
SF1000	208.53	339.25	258.44	+23.8%	0.09424	0.09692

Q21: スケール別実行時間比較

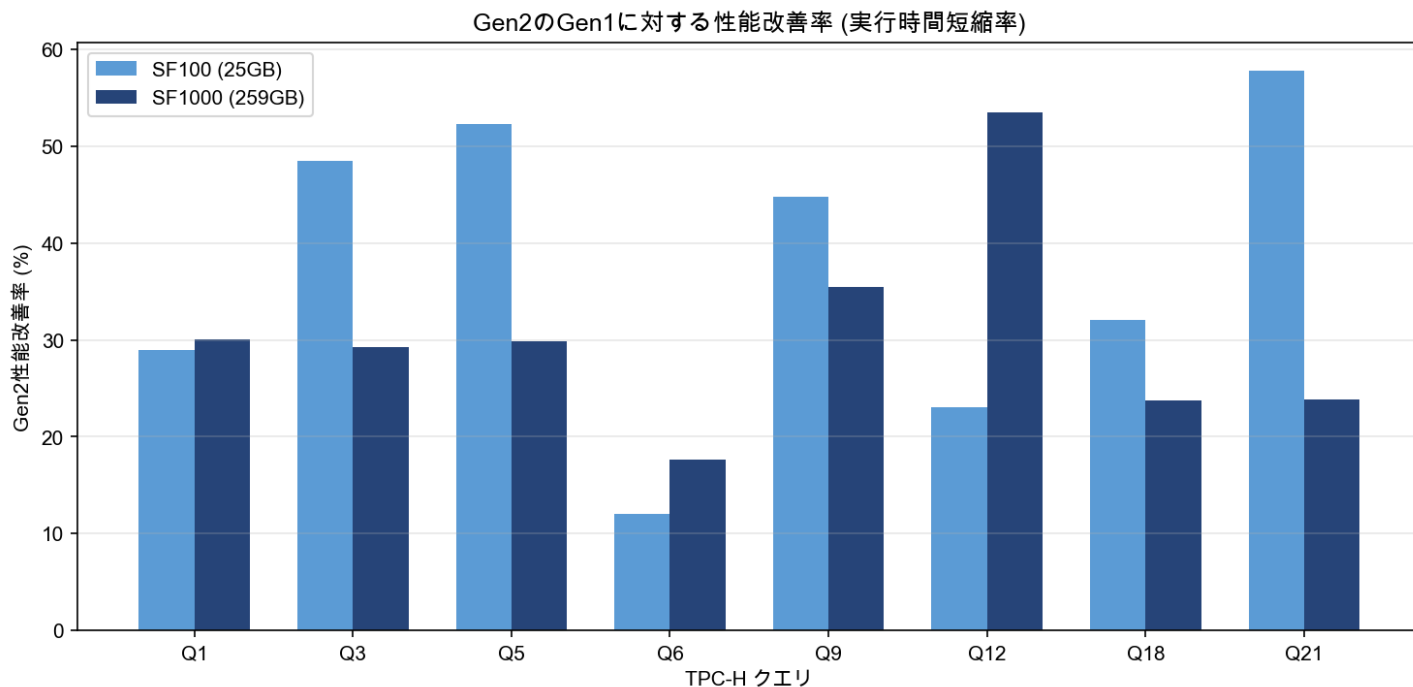


4. まとめ

4.1 全クエリ実行時間比較 (SF1000)



4.2 Gen2性能改善率

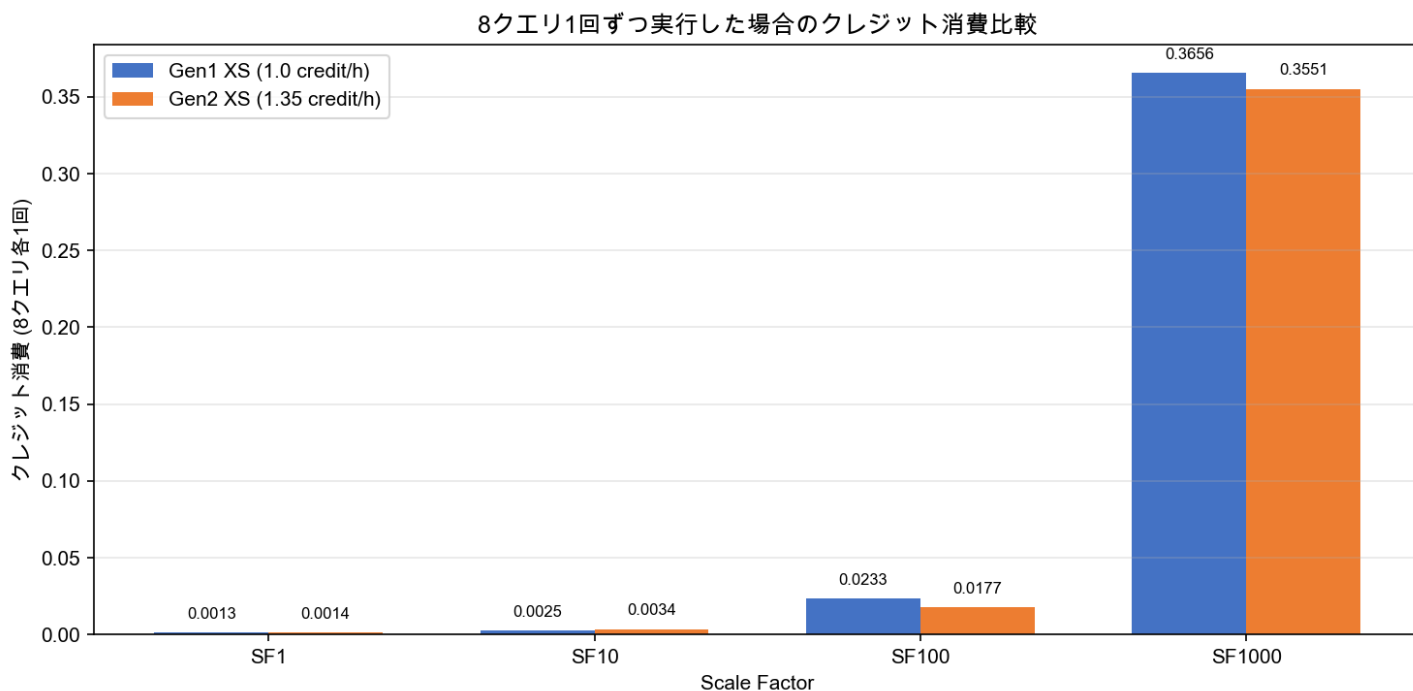


SF100以上の大規模データにおいて、Gen2はGen1に対して一貫して20〜58%の性能改善を達成。

4.3 総合実行時間・クレジット消費比較

Scale	データ量	Gen1合計(秒)	Gen2合計(秒)	時間改善率	Gen1 credit	Gen2 credit	コスト差
SF1	0.243GB	4.7	3.6	+21.9%	0.0013	0.0014	+5.4%
SF10	2.519GB	8.9	9.1	-1.9%	0.0025	0.0034	+37.6%
SF100	24.875GB	83.7	47.2	+43.7%	0.0233	0.0177	-24.0%
SF1000	258.791GB	1316.2	947.0	+28.1%	0.3656	0.3551	-2.9%

4.4 クレジットコスト比較



注: Gen1 XSは1.0 credit/h、Gen2 XSは1.35

credit/hで計算。Gen2は実行時間が短縮されるため、単価が高くても実際のクレジット消費が少なくなるケースがある。

4.5 結論

1. 性能面:

Gen2はSF100(25GB)以上のデータに対して平均37%、SF1000(259GB)では平均30%の実行時間短縮を達成する。特にQ12(53%改善)、Q5/Q3(48-52%改善、SF100)で効果が大きい。

2. コスト面: Gen2はクレジット単価が35%高い(1.35 vs 1.0

credit/h)が、実行時間の短縮により実質的なクレジット消費は大規模データで同等〜やや低い水準。SF1000ではGen2のコストがGen1比で約-3% (時間短縮効果が単価増を相殺)。

3. 小規模データ:

SF1(0.2GB)やSF10(2.5GB)の場合、両世代とも全クエリが2秒以内に完了し、性能差はウォームアップや最低課金(60秒)に対して無視できる。

4. クエリ特性:

単純スキャン(Q6)はデータ量増加に対して最も効率的にスケールし(1000倍データに対し27倍の時間)、複雑なJOIN+サブクエリ(Q21, Q18)は最も影響を受ける(800倍以上)。大規模データでこれらのクエリを頻繁に実行する場合は、ウェアハウスのサイズアップを検討すべき。

5. 推奨:

25GB以上のデータを扱う分析ワークロードではGen2の使用を推奨する。単価増(+35%)を差し引いても、実行時間短縮による生産性向上と実質コストの均衡から、Gen2が有利。

6. DMLワークロードへの期待:

本ベンチマークはSELECT (参照系) クエリのみを対象としており、Gen2の性能改善は20〜58%であった。しかし、Snowflakeの公式ドキュメントではGen2は「DELETE, UPDATE, MERGEなどのDML操作に対する最適化強化」が明記されている。そのため、DML (INSERT/UPDATE/DELETE/MERGE) を主体とするETL・データパイプラインワークロードでは、本レポートのSELECTクエリ以上の性能改善 (= クレジット単価増を上回るコスト効率改善) が期待される。Gen2の真価はDMLワークロードでより顕著に発揮されようと考えられる。